SWE-bench_Multilingual_Agentic

Overview

SWE-bench Multilingual Agentic is the agentic-mode evaluation of SWE-bench Multilingual, a 300-task SWE-bench-style benchmark spanning 42 repositories and 9 programming languages. The model autonomously explores, edits, and submits a patch through a multi-turn agent loop inside a per-instance Docker container.

Task Description

- **Task Type**: Automated Software Engineering / Bug Fixing (Agentic, Multilingual)
- **Input**: GitHub issue description (no oracle file context)
- **Output**: Code patch (diff format) collected from `git diff` after autonomous editing
- **Languages**: C, C++, Go, Java, JavaScript/TypeScript, PHP, Ruby, and Rust

Key Features

- 300 curated Issue-Pull Request tasks
- 42 real-world repositories across 9 programming languages
- Multi-turn agent loop with per-instance SWE-bench Docker s
- SWE-bench-compatible patch evaluation using fail-to-pass and pass-to-pass tests

en latest

Evaluation Notes

- Requires `pip install swebench==4.1.0` before evaluation
- Uses the official SWE-bench Multilingual x86_64 instance images and sets Docker platform to `linux/amd64` automatically
- Docker images are built/pulled automatically for each instance
- Timeout of 1800 seconds (30 min) per instance for final patch validation
- See the usage documentation↗ for detailed setup instructions
- Supports both local image building and remote image pulling

Agentic Mode

This benchmark drives a multi-turn agent loop (mirrors mini-swe-agent's `swebench.yaml`) inside a per-instance SWE-bench Docker container. The model issues `bash` commands to explore `/testbed`, edit source files, and finally submits its `git diff` patch by printing the sentinel `COMPLETE_TASK_AND_SUBMIT_FINAL_OUTPUT` followed by the patch contents.

The default `swe_bench_toolcall` strategy uses OpenAI function calling with a single `bash` tool. Models without function-calling support can select `swe_bench_backticks` through `NativeAgentConfig.strategy`; that strategy expects one ```` ```mswea_bash_command ``` ```` block per turn.

Properties

Property	Value
Benchmark Name	`swe_bench_multilingual_agenti`
Dataset ID	SWE-bench/SWE-bench_Multilingual↗

en latest

Property	Value
Paper	N/A
Tags	`Coding`
Metrics	`accuracy`
Default Shots	0-shot
Evaluation Split	`test`

Data Statistics

Metric	Value
Total Samples	300
Prompt Length (Mean)	2197.94 chars
Prompt Length (Min/Max)	124 / 69351 chars

Sample Example

Subset: `default`


```
{
  "input": [
    {
      "id": "52ece3cf",
      "content": "Support Post aggregation function pow(f1,f2) to cater for squ
    }
  ],
  "id": 0,
  "group_id": 0,
  "tools": [
    {
      "name": "bash",
      "description": "Execute a bash command inside th
      "parameters": {
        "properties": {
```


en latest

```
            "command": {
              "type": "string",
              "description": "The bash command to execute."
            },
            "timeout": {
              "type": "number",
              "description": "Maximum execution time in seconds (default: 60).",
              "default": 60
            }
          },
          "required": [
            "command"
          ]
        }
      }
    ],
    "metadata": {
      "problem_statement": "Support Post aggregation function pow(f1,f2) to cater
      "instance_id": "apache__druid-13704",
      "base_commit": "51dfde02840017092486fb75be2b16566aff6a19",
      "patch": "diff --git a/docs/querying/post-aggregations.md b/docs/querying/p
      "PASS_TO_PASS": [
        "org.apache.druid.query.aggregation.post.ArithmeticPostAggregatorTest#tes
        "org.apache.druid.query.aggregation.post.ArithmeticPostAggregatorTest#tes
      ],
      "FAIL_TO_PASS": [
        "org.apache.druid.query.aggregation.post.ArithmeticPostAggregatorTest#tes
      ],
      "test_patch": "diff --git a/processing/src/test/java/org/apache/druid/query
      "version": "13704",
      "repo": "apache/druid",
      "environment_setup_commit": null,
      "hints_text": "",
      "created_at": "2023-01-23 04:10:47",
      "docker_image": "swebench/sweb.eval.x86_64.apache_1776_druid-13704:latest"
    }
}
```


Prompt Template

Prompt Template:

en latest

```
{question}
```

Extra Parameters

Parameter	Type	Default	Description
`build_docker_images`	`bool`	`True`	Build Docker images locally for each sample.
`pull_remote_images_if_available`	`bool`	`True`	Attempt to pull existing remote Docker images before building.
`force_arch`	`str`	``	Optionally force a specific architecture for image build/pull. Choices: [‘’, ‘arm64’, ‘x86_64’]
`dockerhub_username`	`str`	`swebench`	DockerHub user/org namespace for remote SWE-bench images.

Usage

Using CLI

```
evalscope eval \
    --model YOUR_MODEL \
    --api-url OPENAI_API_COMPAT_URL \
    --api-key EMPTY_TOKEN \
    --datasets swe_bench_multilingual_agentic \
    --agent-config '{"mode":"native","strategy":"swe_bench_toolcall","max_steps
    --limit 10  # Remove this line for formal evaluation
```

Using Python

en latest

```
from evalscope import TaskConfig, run_task
from evalscope.api.agent import NativeAgentConfig
```

```python
task_cfg = TaskConfig(
    model='YOUR_MODEL',
    api_url='OPENAI_API_COMPAT_URL',
    api_key='EMPTY_TOKEN',
    datasets=['swe_bench_multilingual_agentic'],
    agent_config=NativeAgentConfig(
        strategy='swe_bench_toolcall',
        max_steps=250,
    ),
    dataset_args={
        'swe_bench_multilingual_agentic': {
            # extra_params: {}  # uses default extra parameters
        }
    },
    limit=10,  # Remove this line for formal evaluation
)

run_task(task_cfg=task_cfg)
```

‹ SWE-bench_Lite_Agentic

SWE-bench_Pro ›

© 2022-2024, Alibaba ModelScope Built with Sphinx 9.1.0

en latest